# Rules: Soft and Strong Governance

# The Role of Governance Rules

## The Role of Governance Rules I

Research software projects make dozens of implicit governance decisions: what test-coverage threshold is acceptable, how manuscript sections should be ordered, which citation fields are mandatory. Left implicit, these decisions drift silently across projects in a monorepo, eroding the consistency that makes the repository valuable as a public exemplar. The rules layer makes governance explicit, versioned, and machine-enforceable.

A **rule set** in the template repository is a directory under `rules/<scope>/<name>/` containing a typed manifest (`rules.yaml`) and two subdirectories of rule files:

## The Role of Governance Rules

```
<name>/
   rules.yaml          - manifest (type, scope, version,
rule_kinds)
   soft/               - Markdown guideline files
(human-readable, prompt-like)
   strong/             - YAML constraint schemas
(machine-enforceable)
```

This two-tier architecture reflects the distinction between *guidance* (which humans follow approximately) and *constraints* (which pipelines enforce precisely) — a distinction also recognised in enterprise application architecture (Fowler 2002). fig. 3 shows both template rule sets split into their soft and strong branches: each branch is independently discoverable, so a consumer that only cares about machine-enforceable constraints never has to parse guideline prose, and vice versa.

# Soft Rules: Style and Process Guidelines

## Soft Rules: Style and Process Guidelines I

Soft rules are Markdown files in `soft/`. They encode preferences and conventions that cannot easily be expressed as boolean constraints but that human reviewers and AI agents can apply contextually. Examples include:

# Soft Rules: Style and Process Guidelines I

- **Style preferences**: "Prefer active voice in manuscript sections." "Use `\module{}` macros for all code identifiers."
- **Process guidelines**: "Tag pull requests with a review-stage label before requesting review." "Update `TODO.md` before closing an issue."
- **Citation conventions**: "Cite primary sources rather than textbooks where possible."

## Soft Rules: Style and Process Guidelines I

Soft rules are treated as guidance: deviations surface as suggestions in code review and manuscript audit reports, not as pipeline blockers. This makes the soft layer suitable for evolving preferences that should not break automated checks.

# Strong Rules: Hard Constraints

## Strong Rules: Hard Constraints I

Strong rules are YAML files in `strong/`. Each file defines one named constraint:

```yaml
rule:
  name: coverage-gate
  kind: strong
  description: "Minimum test coverage threshold for
src/ modules."
  applies_to: "projects/*/src/"
  enforcement: fail_on_violation
  constraints:
    infrastructure: 60
    project_src: 90
    public_api: 95
```

## Strong Rules: Hard Constraints I

The `enforcement: fail_on_violation` field signals that a pipeline must halt and report when this rule is violated. Strong rules are suitable for invariants that, if broken, indicate a genuine defect rather than a style preference: coverage below the source-declared threshold means tests are missing; a manuscript section without an abstract means the document is incomplete.

# Strong Rules: Hard Constraints I



## Strong Rules: Hard Constraints I

Figure 3: Rule hierarchy: the two template rule sets, each split into a machine-enforceable `strong/` branch and a guidance-only `soft/` branch.

# The Two Template Rule Sets

`template_project_rules` |

This rule set governs software projects throughout the template repository. Its strong rules currently comprise:

# template_project_rules

| File | Constraint |
|---|---|
| strong/coverage-gate.yaml | Minimum coverage: infrastructure 60%, project src/ 90%, public API 95% |
| strong/module-structure.yaml | Required directory layout: src/, tests/, scripts/, manuscript/ |

Its soft rules provide guidance on code style, commit message conventions, and pull-request labelling.

This rule set governs research manuscripts. Its strong rules comprise:

# template_manuscript_rules

| File | Constraint |
|---|---|
| strong/reference-schema.yaml | Required BibTeX fields and cite-key format constraints |
| strong/section-schema.yaml | Required manuscript sections, ordering, and minimum word counts |

In the current pipeline run, **2 of 2 rule sets** validated successfully (fig. 5).

# Rule Application Module

## Rule Application Module I

The src/template_pools_rules_tools/rules/rules_applier.py module exposes three functions:

```
from template_pools_rules_tools.rules.rules_applier
import (
    load_soft_rules,
    load_strong_rules,
    validate_against_rules,
)


soft   = load_soft_rules("template_project_rules")
# list[dict]
strong = load_strong_rules("template_project_rules")
# list[dict]
result =
validate_against_rules("template_project_rules")
# → {"status": "ok" | "partial" | "missing",
"soft_count": N, "strong_count": N}
```

`validate_against_rules()` performs two checks: (1) the `rules.yaml` manifest is parseable YAML; (2) every rule file in `soft/` and `strong/` is parseable YAML. It returns a status of `"ok"` when both checks pass, `"partial"` when the manifest exists but some rule files are missing or malformed, and `"missing"` when the rule set directory is absent entirely. This graduated status enables the integration pipeline to distinguish between a rule set that has not yet been created (acceptable during active development) and one that is present but broken (actionable defect).

# Rules and Manuscript Variables

## Rules and Manuscript Variables I

Strong rule validation counts are injected into the manuscript through the token system. The token 2 expands to the count of rule sets that returned `status="ok"` during the integration run. This creates a verifiable link between the pipeline's actual behaviour and the manuscript's claims — the manuscript cannot assert successful validation without the pipeline having actually succeeded.

# Beyond Structural Validation: Semantic Rule Evaluation

## Beyond Structural Validation: Semantic Rule Evaluation I

`validate_against_rules()` (described above) performs *structural* validation only: it confirms that `rules.yaml` and every file in `soft//strong/` parse as YAML. It does not check whether the constraints those strong-rule files declare are actually satisfied by the current project. That semantic layer lives in a separate module, `src/template_pools_rules_tools/rules/strong_rule_evaluator.py`, exposed via `scripts/04_validate_strong_rules.py`:

## Beyond Structural Validation: Semantic Rule Evaluation I

```
from template_pools_rules_tools.rules.strong_rule_eva
luator import evaluate_strong_rules,
load_rule_context_from_project

context =
load_rule_context_from_project(project_root)
result =
evaluate_strong_rules("template_project_rules",
context)
# → {"rule_set": ..., "evaluations": [...], "passed":
bool, "violation_count": int}
```

## Beyond Structural Validation: Semantic Rule Evaluation I

`evaluate_strong_rules()` dispatches each strong-rule YAML file to a rule-kind-specific evaluator function keyed by its declared kind, via a small dispatch table covering all four strong-rule kinds that exist across both rule sets:

# Beyond Structural Validation: Semantic Rule Evaluation I

| Kind | Evaluator | Checks |
|---|---|---|
| `coverage_threshold` | `_evaluate_coverage_threshold` | Measured coverage percentages (from `context["coverage"]`) against each constraint's declared `minimum_line_coverag e` |
| `module_structure` | `_evaluate_module_structure` | Required project directory layout (`src/`, `tests/`, `scripts/`, `manuscript/`) actually exists |
| `section_schema` | `_evaluate_section_schema` | Required manuscript sections, ordering, and forbidden placeholder headings (`TODO`, `Draft`, etc.) |
| `reference_schema` | `_evaluate_reference_schema` | Required BibTeX fields and cite-key format constraints on every parsed reference entry |

## Beyond Structural Validation: Semantic Rule Evaluation I

Each evaluator distinguishes structured violation *reasons* rather than collapsing everything to a boolean — for example, `coverage_threshold` reports separately whether a key was absent from context (a context-completeness issue), non-numeric (a context-shape issue), or numeric-but-below-minimum (a genuine rule violation). This is what lets the pipeline tell a maintainer *why* a rule failed, not merely *that* it failed — directly addressing the "actionable defect" distinction introduced in the previous section.

## Beyond Structural Validation: Semantic Rule Evaluation I

Crucially, `section_schema` and `reference_schema` are not evaluated against synthetic fixtures — `load_rule_context_from_project()` (in `scripts/04_validate_strong_rules.py`) builds their context by parsing *this project's own, current* `manuscript/references.bib` into structured reference entries and extracting the real #-level headings from every file under `manuscript/*.md`. Running `uv run python projects/templates/template_pools_rules_tools/scripts/04_validate_strong_rules.py` therefore semantically validates this exact manuscript's own bibliography and section structure, live, on every invocation — re-run that command to see the current evaluation and violation counts rather than trusting a number printed here, since either count can legitimately change as the manuscript grows.